INFRADESK

Offline IT Asset Management and Problem Reporting System with DevOps Automation

PROJECT SYNOPSIS

Submitted in partial fulfilment of the requirements of the course

[Course name and code]

Student Name	**[Your full name]**
Registration No.	**[Your registration / roll number]**
Programme / Branch	**[Programme, branch, semester]**
Institution	**[College / university name]**
Guide / Instructor	**[Instructor name]**
Date of Submission	October 2026

Items in square brackets must be completed by the student before submission.

Contents

1 Abstract **3**

2 Introduction and Background **3**

3 Problem Statement **3**

4 Objectives **4**

5 Scope and Limitations **4**

5.1 In scope 4

5.2 Out of scope 4

5.3 Security notice 5

6 Proposed System **5**

6.1 Users and Roles 5

6.2 Main Modules 5

6.3 Ticket Status Workflow 6

7 System Architecture **6**

7.1 Division of Responsibility 7

8 Technology Stack **7**

9 Database Design **8**

10 Methodology **8**

11 Testing and Evaluation Plan **9**

12 Hardware and Software Requirements **9**

13 Expected Outcomes **10**

14 Future Enhancements **10**

15 Project Plan (Estimate) **10**

16 References **10**

1 Abstract

Small laboratories, student project teams and local IT support groups often record technical problems on paper, in chat messages or in spreadsheets. Such records are easily lost, assignments are unclear and nobody can say who is working on which fault. At the same time, an application that is installed by hand on one computer is difficult to reproduce and may fail without anyone noticing.

InfraDesk is a locally hosted web application in which an administrator manages users, assets and tickets while workers report problems, receive assignments, add work notes and resolve tickets. All data is stored in a local SQLite database, and no paid or cloud service is needed at run time. The application is packaged in Docker containers and surrounded by a DevOps toolchain with clearly separated responsibilities: **Terraform** declares the local Docker infrastructure, **Puppet** enforces the configuration of an Ubuntu host, **Ansible** deploys and validates the application, and **Nagios Core** monitors its health endpoint and reports failures.

The project therefore combines a practical full-stack application with a reproducible deployment and monitoring workflow.

2 Introduction and Background

IT support is a routine need in computer laboratories, hostels, small offices and student teams. Commercial IT service management products exist, but they are costly, heavy to install and far larger than the needs of a small group. Free alternatives often require a database server, a mail server or a cloud account.

DevOps practice adds a second concern: how the application is installed, configured, updated and watched. Infrastructure as Code, configuration management, automated deployment and monitoring are normally taught separately. InfraDesk uses one small but real application as the subject of all four, so that each tool has a concrete and demonstrable purpose.

3 Problem Statement

1. Technical problems are reported informally, so there is no reliable history, ownership or status.

2. There is no simple, free, offline tool for a small group to track tickets and local equipment.

3. Manual installation is inconsistent from one machine to another and difficult to repeat.

4. Host configuration can drift away from the intended state without being noticed.
5. A stopped or unhealthy application is detected only when a user complains.

4 Objectives

1. Build a role-based web application (Admin and Worker) for reporting, assigning, tracking and resolving IT support tickets.
2. Maintain a simple, manually entered register of local assets and allow tickets to refer to them.
3. Provide in-app notifications, ticket history, dashboards, reports and CSV export computed from real database records.
4. Run entirely on one computer without internet access once dependencies have been downloaded.
5. Package the system in Docker and provide a repeatable deployment.
6. Use Terraform, Puppet, Ansible and Nagios Core for distinct, documented and demonstrable DevOps tasks.
7. Verify the system with automated tests and report only results that were actually obtained.

5 Scope and Limitations

5.1 In scope

Login and role-based authorization; ticket creation, assignment, status workflow, comments and history; in-app notifications; user and asset management; reports and CSV export; health and readiness endpoints; backup and restore of the database; Docker packaging; and the four DevOps components listed above.

5.2 Out of scope

InfraDesk is a student prototype and a local support tool. It is *not* an emergency dispatch system, a production ITSM product or a remote monitoring agent. It does not discover devices automatically, does not send e-mail, SMS or push notifications, and does not claim that every failure is repaired automatically. Nagios detects and reports failures; recovery is performed by a documented, human-triggered Ansible procedure unless a tested event handler is explicitly added.

5.3 Security notice

The prototype is intended for a trusted local network. It must not be exposed directly to the public internet without a security review. SQLite is suitable for this single-computer prototype; a production deployment with many concurrent users would require a server database such as PostgreSQL.

6 Proposed System

6.1 Users and Roles

Role	Permissions
Admin	Manage users, tickets, assignments, priorities and statuses; maintain the asset register; view reports, activity history and CSV export.
Worker	Report problems, see own and assigned tickets, add work notes, update permitted statuses, resolve tickets with a resolution note, read notifications.

All permissions are enforced on the backend; the frontend only reflects them.

6.2 Main Modules

- **Authentication:** salted password hashing, short-lived signed tokens, inactive-account blocking.
- **Ticket workflow:** statuses OPEN, ASSIGNED, IN_PROGRESS, WAITING, RESOLVED, CLOSED and CANCELLED with validated transitions; priorities LOW to URGENT; categories such as Hardware, Software, Network, Printer and Account Access.
- **Comments and history:** every comment, assignment and status change is stored as an event.
- **Notifications:** in-app, generated by real actions such as assignment, comment and status change.
- **Asset register:** asset tag, name, type, location, serial number, status and notes.
- **Dashboards and reports:** counts by status, priority, category and assignee computed from the database.
- **Operations:** `/health` and `/ready` endpoints, backup and restore scripts.

6.3 Ticket Status Workflow

From	Allowed next status
OPEN	ASSIGNED, IN_PROGRESS, CANCELLED
ASSIGNED	IN_PROGRESS, WAITING, CANCELLED
IN_PROGRESS	WAITING, RESOLVED
WAITING	IN_PROGRESS, RESOLVED
RESOLVED	CLOSED, IN_PROGRESS (reopen)
CLOSED	IN_PROGRESS (reopen, admin only)

7 System Architecture

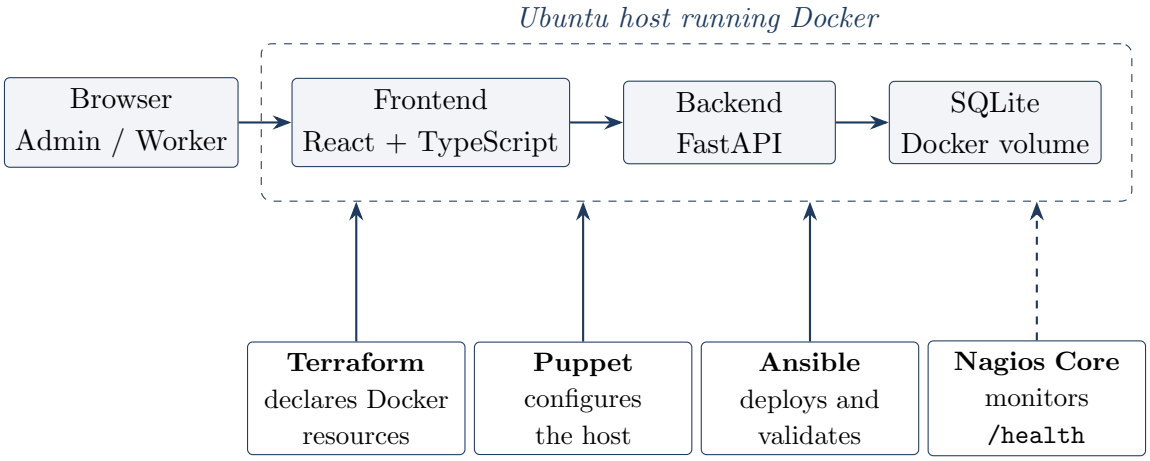


Figure 1. The top row shows the request path of the application. Each tool in the bottom row acts on the Ubuntu and Docker environment that hosts it; the dashed arrow marks monitoring, which only observes and reports.

7.1 Division of Responsibility

Tool	Responsibility	Demonstration
Docker Compose	Simple development and demo run	Start application with a few commands; data survives restarts
Terraform	Declares the local Docker network, volume and containers in its own deployment mode	`init`, `validate`, `plan`, `apply`, `destroy`
Puppet	Baseline of the Ubuntu host: packages, directories, configuration files, service	Run twice for idempotence; change a managed file, re-run, show correction
Ansible	Preflight checks, deployment, validation, recovery	Playbook runs with handlers and a health check
Nagios Core	Monitors host and the `/health` endpoint	OK → CRITICAL when stopped → OK after recovery

Terraform and Docker Compose are separate modes and never manage the same containers at the same time.

8 Technology Stack

Layer	Technology
Frontend	React, TypeScript, Vite, plain CSS (no external CDN, no icon library)
Backend	Python, FastAPI, Pydantic, SQLAlchemy, Alembic, Uvicorn
Database	SQLite on a persistent Docker volume
Security	Password hashing, signed tokens, backend role checks, narrow CORS
Packaging	Docker, Docker Compose
Infrastructure as Code	Terraform (Docker provider)
Configuration management	Puppet (`puppet apply` on an Ubuntu host or VM)
Deployment automation	Ansible
Monitoring	Nagios Core
Testing	pytest, Vitest and React Testing Library
Source control	Git and GitHub

All tools are free and open source. No paid cloud service, paid API, map, SMS or e-mail service is required.

9 Database Design

The core tables are listed below.

Table	Purpose
`users`	Accounts, hashed passwords, role, active flag
`tickets`	Support tickets with readable number (for example HD-000001), status, priority, category, reporter, assignee, optional asset
`ticket_comments`	Work notes and comments
`ticket_history`	Event log of assignments, status changes and comments
`notifications`	In-app notifications with read state
`assets`	Local equipment register

Timestamps are stored in UTC. Foreign keys protect history; records referenced by history are archived or retired rather than deleted.

10 Methodology

Development follows twelve ordered phases. The application must work after every phase before the next one starts.

1. Foundation: repository, configuration, database, health endpoints.
2. Authentication and role-based authorization, with tests.
3. Ticket workflow, comments, history and notifications, with tests.
4. User management, assets, reports and CSV export.
5. Responsive frontend connected to the real API.
6. Docker images and Compose with persistent storage.
7. Terraform deployment mode.
8. Puppet host configuration.
9. Ansible preflight, deploy, validate and recovery.
10. Nagios monitoring and failure demonstration.

11. Offline operation, backup and restore verification.

12. Documentation, report, demonstration script and viva preparation.

11 Testing and Evaluation Plan

- **Backend tests:** login, inactive users, role boundaries, ticket persistence, transition rules, notifications, assets, reports, CSV escaping, input validation.
- **Frontend tests:** login validation, protected routes, role-aware navigation, form validation, loading, error and empty states.
- **DevOps checks:** Docker build, Compose validation, `terraform validate`, Puppet syntax check and repeat run, `ansible-playbook --syntax-check`, Nagios configuration test.
- **Failure demonstration:** stop the application and confirm that Nagios reports CRITICAL, then restore and confirm OK.
- **Operational checks:** backup and restore of a disposable database; operation with the internet disconnected after dependencies have been cached.

Results will be recorded exactly as obtained. A check that cannot be run in a given environment will be listed as *not run* together with the reason.

12 Hardware and Software Requirements

Item	Requirement
Computer	Windows 11 laptop, 8 GB RAM, Ryzen 5 class CPU; no GPU needed
Operating systems	Windows 11 with Docker Desktop (WSL2), and Ubuntu 22.04/24.04 (VM or native) for the DevOps demonstration
Software	Docker, Git, Python 3.11+, Node.js 20+, Terraform, Puppet, Ansible, Nagios Core
Network	Internet only for first-time downloads; afterwards a local network is sufficient

13 Expected Outcomes

- A working offline ticketing and asset register application with Admin and Worker portals.
- A repository that can be started with a small number of documented commands.
- Four demonstrable DevOps components with clear ownership boundaries.
- Automated test suites and honest, reproducible test records.
- Documentation: architecture, workflow, database design, API reference, security notes, installation guides, troubleshooting, demonstration script and viva preparation.

14 Future Enhancements

Optional e-mail notification through a local mail server, PostgreSQL for multi-user production use, file attachments with validation, rate limiting, and a tested Nagios event handler for narrowly scoped automatic recovery.

15 Project Plan (Estimate)

Phase	Activity	Estimated effort
1–2	Foundation, authentication, roles	**[days]**
3–4	Tickets, users, assets, reports	**[days]**
5	Frontend	**[days]**
6–7	Docker and Terraform	**[days]**
8–10	Puppet, Ansible, Nagios	**[days]**
11–12	Offline checks, documentation, report	**[days]**

Durations are to be estimated by the student according to the available schedule.

16 References

[1] FastAPI documentation. https://fastapi.tiangolo.com

[2] SQLAlchemy documentation. https://docs.sqlalchemy.org

[3] React documentation. https://react.dev

[4] Docker documentation. https://docs.docker.com

[5] Terraform documentation, Docker provider. https://developer.hashicorp.com/terraform/docs

[6] Puppet documentation. https://www.puppet.com/docs/puppet

[7] Ansible documentation. https://docs.ansible.com

[8] Nagios Core documentation. https://www.nagios.org/documentation

Signature of Student: _____

Date: _____

Signature of Guide: _____

Date: _____